

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.M71-мм

Анализ подходов к разработке BPU и архитектуры ядра scr1

Бикеев Кирилл Алексеевич

Отчёт по производственной практике
в форме «Решение»

Научный руководитель:
ст. преподаватель кафедры ИАС, К. К. Смирнов

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Branch Prediction Unit	5
2.1. Статические методы предсказания ветвлений	5
2.2. Динамические методы предсказания ветвлений	6
3. Оценка влияния BPU на характеристики ядра	9
3.1. Общие количественные ориентиры из литературы	9
3.2. Реализации BPU в открытых RISC-V-совместимых ядрах MCU-класса	10
4. Практическая часть работы	14
4.1. Какие данные возможно получать	14
4.2. Реализация дампа счётчиков	15
Заключение	17
Список литературы	19

Введение

Архитектура RISC-V сегодня набирает всё большую популярность за счёт своей гибкости и открытости. В международный альянс RISC-V входят такие гиганты, как Google, nvidia, Huawei, Qualcomm [1]. В России также присутствует немалый интерес к данной архитектуре. Например, компания Syntacore разработала линейку ядер на базе RISC-V. Самое младшее из них — scr1 — является предметом исследования данной работы, поскольку имеет широкий спектр применения и открытую архитектуру [2].

scr1 является ядром MCU-класса [2], следовательно, оптимизировано под уменьшение площади и энергопотребления, вследствие чего его текущая реализация не предусматривает наличия BPU (Branch Prediction Unit) [3]. Однако нельзя утверждать, что в ядрах MCU-класса BPU не реализуется в принципе. Например, чипы на базе ARM Cortex-M7 обладают модулем предсказаний [4], ядра RISC-V-совместимых чипов, например Ibex, также обладают BPU [13]. В данном случае актуален вопрос «какой подход к реализации BPU стоит выбрать, чтобы добиться оптимального соотношения прироста производительности к приросту энергопотребления и площади?»

Чтобы дать ответ на столь комплексный вопрос, необходимо детально погрузиться в архитектуру scr1 и подходы к реализации и интеграции BPU, а также провести серию экспериментов и выработать методику оценки качества. Данная работа будет сфокусирована на погружении в архитектуру и подходы.

1. Постановка задачи

Целью работы является изучение подходов к реализации BPU и архитектуры scr1. Для её выполнения были поставлены следующие задачи:

1. сделать обзор подходов к реализации BPU;
2. сделать обзор существующих open-source RISC-V-совместимых архитектур с точки зрения реализации и внедрения BPU;
3. предложить оптимальные подходы для scr1;
4. реализовать дамп счётчиков scr1.

2. Branch Prediction Unit

Ядро `scr1` использует 2–4-ступенчатый конвейер (в зависимости от заданной спецификации), следовательно, для него, как и для любого другого ядра, использующего конвейер, актуальна проблема наличия накладных расходов при обработке ветвлений. На рисунке 1 показан абстрактный 2-ступенчатый конвейер: вторая часть инструкции *A* выполняется одновременно с первой частью инструкции *B* и так далее. Однако в случае, когда необходимо обработать ветвление, следующая инструкция заранее неизвестна, из-за чего все преимущества конвейера теряются на конкретном участке кода. В современных программах ветвлений могут быть тысячи, поэтому накладные расходы высоки. Чтобы этого избежать, был придуман механизм предсказания ветвлений — BPU. Обзору подходов к его реализации будет посвящён данный параграф, основанный на материалах из книги «Modern Processor Design: Fundamentals of Superscalar Processors» [10].

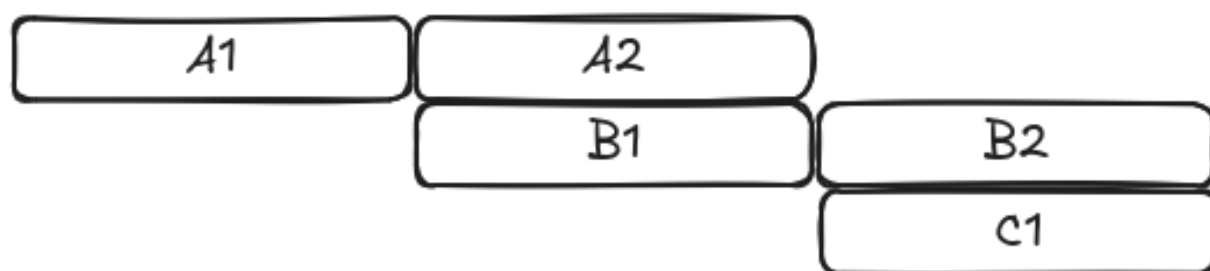


Рис. 1: Абстрактный 2-ступенчатый конвейер

2.1. Статические методы предсказания ветвлений

Самая простая стратегия предсказания ветвлений состоит в том, чтобы предсказать, что все ветви всегда будут идти в одном и том же направлении (всегда выбранном или невыбранном). Стратегия «*always not taken*» предполагает, что ветвь никогда не будет выбрана, поэтому всегда начинает выполняться инструкция, следующая за текущей. Однако, как показывает статистика, в 60% случаев ветвь берётся, поэтому стратегия «*always taken*» считается более эффективной. В текущей

реализации `scr1` используется стратегия «always not taken».

Логическим продолжением упомянутого выше является стратегия «backwards taken/forwards not-taken» (BTFNT). Логическим обоснованием этой эвристики является то, что большинство обратных ветвей являются ветвями цикла, и поскольку циклы обычно повторяются много раз перед выходом, эти ветви, скорее всего, будут приняты. Такую стратегию, например, использовал процессор Intel Pentium 4 в качестве резервной, когда его динамический предиктор не мог дать предсказания.

2.2. Динамические методы предсказания ветвлений

В отличие от статических схем, решение которых фиксировано на этапе проектирования и не зависит от истории выполнения, динамическое предсказание ветвлений принимает решение во время работы, опираясь на накопленную историю поведения переходов. Это снимает фундаментальный потолок статических методов — неспособность адаптироваться к данным: один и тот же переход, поведение которого меняется в ходе исполнения программы, при динамическом подходе предсказывается по-разному в зависимости от наблюдаемой динамики. Платой за это служит аппаратная сложность: динамический предсказатель требует таблиц состояния, регистров истории и логики обновления, что увеличивает площадь, энергопотребление и потенциально удлиняет критический путь стадии выборки.

Простейшая динамическая схема хранит таблицу счётчиков, индексируемую младшими битами адреса перехода (PC). Каждая ячейка — это двухбитный насыщающий счётчик с четырьмя состояниями (сильно/слабо «переход произойдёт», слабо/сильно «не произойдёт»). Предсказание определяется старшим битом счётчика, а после разрешения перехода счётчик инкрементируется при выполненном переходе и декрементируется при невыполненном, не выходя за границы насыщения. Двухбитная организация (в отличие от однобитной) обеспечивает гистерезис: единичная аномалия в поведении регулярного перехода не приво-

дит к немедленной смене предсказания, что особенно ценно для циклов, где переход замыкания ошибается лишь однократно на выходе. Такая схема дешёва и даёт точность порядка 80–90% на типичных нагрузках, но не учитывает корреляцию между разными переходами.

Ключевое наблюдение состоит в том, что исход перехода часто коррелирует с исходами предшествующих переходов. Это привело к двухуровневой адаптивной схеме, состоящей из двух структур: регистра истории ветвлений (Branch History Register, BHR), накапливающего последовательность последних исходов в виде сдвигового регистра, и таблицы образцов (Pattern History Table, PHT) из насыщающих счётчиков. Первый уровень (история) формирует индекс во второй уровень (таблицу счётчиков), и именно выбранный счётчик даёт предсказание. Тем самым предсказание обусловлено не только адресом перехода, но и контекстом — образцом недавнего поведения потока управления.

Двухуровневые схемы классифицируются по тому, как организованы оба уровня. История может быть глобальной (один общий BHR на все переходы, обозначается G), индивидуальной по адресу (отдельный BHR на каждый переход, обозначается P — per-address) или групповой (BHR на множество переходов, S — set). Аналогично таблица PHT может быть глобальной (g), индивидуальной (p) или групповой (s). Отсюда семейство обозначений — GAg, GAp, PAg, PAp, SAg и т. д., где первая буква описывает организацию истории, вторая (A) указывает на адаптивность, третья — организацию таблицы. Глобальная история эффективна там, где переходы коррелируют между собой; индивидуальная — там, где важна собственная динамика конкретного перехода (например, чередующиеся образцы внутри одного цикла).

Прямое использование глобальной истории как индекса (схема, исследованная Pan, So, Rahmeh, 1992) приводит к тому, что многие переходы делят одни и те же ячейки PHT. McFarling (1993) предложил два экономичных способа объединить адрес и историю в один индекс. В схеме gselect биты PC и биты истории конкатенируются, образуя составной индекс. В более удачной схеме gshare биты PC и биты глобальной истории комбинируются операцией «исключающее ИЛИ» (XOR),

что при той же длине индекса позволяет различать больше комбинаций «адрес + контекст» и лучше распределяет обращения по таблице. gshare стал де-факто эталонной базовой схемой, относительно которой принято сравнивать более сложные предсказатели.

3. Оценка влияния ВРУ на характеристики ядра

3.1. Общие количественные ориентиры из литературы

Прежде чем оценивать применимость ВРУ к ядру `scr1`, целесообразно обозначить количественные границы, в которых блок предсказания переходов влияет на площадь кристалла и энергопотребление. В литературе эти границы существенно различаются для процессоров высокого класса и для `embedded`-ядер MCU-класса, что напрямую определяет пространство допустимых проектных решений для `scr1`.

Для высокопроизводительных процессоров сложные предсказатели, состоящие из множества крупных структур (РНТ, ВТВ, RAS, таблицы тегов), в совокупности потребляют заметную долю мощности кристалла — в ряде случаев более 10% [6, 9]. При этом необходимо различать вклад собственно предсказателя направления и вклад буфера цели перехода: по оценке Парикха и Скадрона, предсказатель направления сам по себе рассеивает менее 1% полной мощности процессора, тогда как блок предсказания в целом (включая ВТВ, обращение к которому происходит каждый такт) типично составляет около 7% и достигает 10% суммарного энергопотребления [9]. Более агрессивная оценка приводится в работах по статическому планированию ветвлений для встраиваемых процессоров: динамические предсказатели могут составлять от 10% до 40% динамической мощности процессора, причём эта величина пропорциональна числу обращений к предсказателю за время исполнения программы [5].

В отличие от серверных и настольных процессоров, встраиваемые ядра жёстко ограничены по площади кремния и потреблению. Объём оперативной памяти, который может быть выделен под предсказатель, как правило, не превышает 32 Кбит, а нередко составляет лишь 8 Кбит и менее; вследствие этого проектировщик вынужден искать компромисс между ёмкостью таблиц и точностью предсказания [6]. Именно из-за

этого ограничения для встраиваемых процессоров предпочтение отдаётся статическим схемам либо простым динамическим методам уровня gshare, при которых малая ёмкость РНТ повышает риск конфликтов индексов (aliasing) [6].

Количественную оценку компромисса «точность–площадь–мощность» даёт design-space exploration предсказателя TAGE для встраиваемых ядер [6]. При объёме RAM 32 Кбит и регистре глобальной истории (GHR) длиной 64 бита оптимизация по площади (area-first) приводит к потере точности предсказания на 1,89% по сравнению с оптимизацией по точности (accuracy-first), однако обеспечивает сокращение площади на 10,44% и экономию мощности на 11,6%. Кроме того, в той же работе показано, что предсказатель TAGE ёмкостью 1,5 Кбайт по точности сопоставим с GShare ёмкостью 4 Кбайт, а вариант TAGE ёмкостью 0,25 Кбайт практически не уступает 4-килобайтному GShare [6]. Это подтверждает, что для жёстко ограниченных по ресурсам ядер выбор схемы предсказания важнее, чем простое наращивание ёмкости таблиц.

Приведённые ориентиры показывают, что современные продвинутые предсказатели зачастую оказываются слишком велики и энергозатратны, чтобы быть жизнеспособным решением для небольших встраиваемых процессоров, где площадь кристалла, энергопотребление и производительность одновременно находятся под давлением [6, 9]. Применительно к scrl это означает, что добавление полноценного динамического BPU с ВТВ и многотабличной структурой TAGE нецелесообразно без пересмотра всего бюджета площади ядра; реалистичной альтернативой является лёгкий статический предсказатель либо минимальная схема bimodal/gshare с ограниченной ёмкостью РНТ.

3.2. Реализации BPU в открытых RISC-V-совместимых ядрах MCU-класса

В данном разделе рассматриваются конкретные реализации блока предсказания переходов (BPU) в открытых RISC-V-совместимых про-

цессорных ядрах, близких к scr1 по классу (MCU-class, RV32, in-order). Анализ показывает, что пространство проектных решений простирается от полного отсутствия BPU до опционально подключаемых статических и простых динамических предсказателей; при этом полноценные многотабличные схемы (TAGE, tournament) встречаются лишь в ядрах прикладного класса, выходящих за рамки MCU-сегмента.

Ядро scr1, как уже ранее было упомянуто, представляет собой 32-разрядный RISC-V-совместимый процессор MCU-класса с конвейером глубиной от 2 до 4 ступеней в зависимости от конфигурации сборки, реализующий базовый набор RV32I/RV32E с опциональными расширениями M и C [3]. В составе ядра отсутствует динамический BPU в любой из predetermined конфигураций (MAX, BASE, MIN); обработка переходов опирается на статическую стратегию, заложенную в логику конвейера. Именно это обстоятельство является исходной точкой настоящей работы и определяет постановку задачи об оценке применимости BPU к scr1.

Ibex — наиболее близкий к scr1 по архитектуре аналог: это 32-разрядное in-order ядро промышленного качества на языке SystemVerilog, ориентированное на управляющие embedded-приложения и прошедшее несколько циклов кремниевой верификации [13]. Базовая конфигурация Ibex имеет плоский 2-ступенчатый конвейер, к которому опционально подключаются три механизма повышения производительности обработки переходов [7]:

- *Branch Target ALU (BT-ALU)* — отдельный сумматор для параллельного вычисления адреса цели перехода и условия ветвления, сокращающий задержку при неправильно предсказанных переходах;
- *Static Branch Predictor (SBP)* — статический предсказатель, который распознаёт инструкции переходов и предсказывает все безусловные переходы и условные переходы назад как «взятые», а условные переходы вперёд — как «невзятые» (схема BTFNT);
- *Writeback Stage* — дополнительная ступень конвейера для ускоре-

ния транзакций с памятью (приводится здесь для сопоставления масштаба накладных расходов).

Реализация SBP в RTL выполнена через параметр `BranchPredictor`; при его отключении соответствующий путь в стадии выборки синтезируется как недостижимый код и устраняется оптимизатором [7]. Ibex, таким образом, служит эталонным примером того, как статический BPU может быть встроен в ядро масштаба `scr1` в виде опционального параметризуемого модуля.

Ядро CV32E40P (изначально RI5CY, разработанное в рамках PULP Platform и переданное в OpenHW Group) — компактное и энергоэффективное 4-ступенчатое in-order RISC-V-ядро, реализующее RV32IM[F]C с расширениями PULP [7]. В стадии выборки CV32E40P содержит механизм предсказания переходов; в сравнительном анализе с Ibex показано, что наиболее производительные конфигурации Ibex (с BT-ALU и SBP) улучшают задержку команд переходов до уровня, сопоставимого с CV32E40P, при существенно меньшей площади [7].

VexRiscv — модульное FPGA-ориентированное 32-разрядное RISC-V-ядро, в котором функциональность подключается через подключаемые модули (plugin). Механизм предсказания реализован в виде `BranchPlugin` и использует прямо-отображённый буфер цели перехода (BTB) в стадии выборки, хранящий PC инструкции, целевой PC и 2-битный счётчик истории; при правильном предсказании штраф ветвления равен нулю, однако предсказатель создаёт длинный комбинационный путь от порта чтения prediction cache к счётчику команд [11]. В сравнительных исследованиях конфигурация VexRiscv с бимодальным предсказателем и BTB (обозначаемая VR-bp-c) использовалась как эталон динамического предсказания: при общем объёме блочной памяти 4 КБ она реализует 512 записей BTB, что сопоставимо по ресурсам с gshare-предсказателем, имеющим 512 записей BTB и 8192 записи РНТ [8]. Таким образом, VexRiscv демонстрирует пример простого динамического BPU, пригодного для встраиваемых ядер при умеренном бюджете памяти.

Ядро CVA6 (ранее Ariane) относится к прикладному классу: это 6-

ступенчатый in-order процессор, способный загружать ОС семейства Unix. Оно приводится здесь как верхняя граница сложности BPU в открытой экосистеме RISC-V. CVA6 содержит три механизма предсказания следующего адреса: таблицу истории переходов (BHT) на 2-битных счётчиках насыщения, буфер цели перехода (BTV) и стек адресов возврата (RAS); при отсутствии валидного предсказания применяется статическая схема BTFNT, определяемая знаковым битом непосредственного смещения [12]. Для тейпаута в технологии 22 нм FDSOI использовались малые конфигурации структур предсказания (единицы записей), поскольку критический путь ядра проходит через кэш-память, а не через BPU; авторы отмечают, что точность предсказания может быть повышена более сложными схемами (gshare, loop predictor, tournament), однако это направление в их работе не исследовалось [12]. Развитость BPU в CVA6 обусловлена большей глубиной конвейера и, соответственно, более высокой стоимостью неправильного предсказания — фактором, который в ядрах масштаба scr1 выражен существенно слабее.

Сопоставление рассмотренных реализаций позволяет выделить устойчивую закономерность: в открытых RISC-V-ядрах MCU-класса BPU либо отсутствует полностью (scr1), либо реализуется в виде опционального статического предсказателя (Ibex SBP, CV32E40P) или простой динамической схемы с BTV (VexRiscv). Полноценные динамические предсказатели с несколькими таблицами появляются лишь при переходе к прикладному классу с более глубоким конвейером (CVA6). scr1 при этом находится в той же весовой категории, что и Ibex, но, в отличие от последнего, лишён даже опционального статического предсказателя, что и обосновывает исследовательский вопрос настоящей работы.

4. Практическая часть работы

Настоящая работа рассчитана на несколько семестров, и её конечной целью является эмпирическая оценка целесообразности внедрения BPU в ядро `scr1`. Подобная оценка невозможна без измерительной инфраструктуры, способной извлекать из работающего ядра количественные характеристики исполнения — в первую очередь данные о ветвлениях и о цене их обработки. Поэтому экспериментальная часть текущего этапа сосредоточена не на самом сравнении предсказателей (для которого требуется уже синтезированный BPU), а на создании этой инфраструктуры: в загрузочную прошивку `scr1` был реализован *дамп аппаратных счётчиков*.

4.1. Какие данные возможно получать

Источником количественных характеристик исполнения в RISC-V служат аппаратные счётчики, доступные программно через регистры состояния и управления (CSR). Привилегированная спецификация RISC-V определяет три базовых архитектурных счётчика: `cycle` — число тактов с момента сброса, `instret` — число фактически завершённых («retired») инструкций, и `time` — монотонное реальное время, отсчитываемое от системного таймера `mtime`. На 32-разрядных ядрах каждый из них является 64-разрядным и отображается на пару CSR: младшее и старшее слова доступны по отдельным адресам (например, `mcycle` и `mcycleh`). Ядро `scr1` реализует именно этот минимальный архитектурный набор.

Из этого следует характер данных, которые можно извлечь из работающего ядра на текущем этапе. Доступны агрегированные по всей программе величины: суммарное число тактов, число завершённых инструкций и истёкшее реальное время. На их основе вычисляется ключевая интегральная метрика производительности — среднее число тактов на инструкцию $CPI = mcycle/minstret$. Прямой подсчёт переходов и неверных предсказаний недоступен, поскольку для этого требовались бы счётчики событий `mhpcounter`, отсутствующие в `scr1`. Тем не менее

даже базового набора достаточно для целей работы: неверно предсказанные переходы проявляются как дополнительные такты при неизменном числе завершённых инструкций, поэтому влияние будущего BPU на производительность наблюдаемо как изменение CPI одного и того же набора тестовых программ. Измеренные на текущем этапе значения CPI образуют базовую («до BPU») точку отсчёта, относительно которой будет оцениваться эффект от внедрения блока предсказания. Расширение наблюдаемости до пособытийного подсчёта ветвлений (добавление пользовательских счётчиков либо задействование механизма `mhpmevent`) выделяется как направление последующих этапов работы.

4.2. Реализация дампа счётчиков

Считывание счётчиков выполнено на уровне загрузочной прошивки `sc-bl` (First Stage Boot Loader) — штатного начального загрузчика `scr1` [14]. Такой выбор точки внедрения обусловлен тем, что прошивка выполняется непосредственно на исследуемом ядре в машинном режиме и потому обладает прямым доступом к машинным CSR-счётчикам без участия операционной системы или отладчика; при этом сам загрузчик уже располагает инициализированным каналом UART, пригодным для вывода результатов. Реализация оформлена как Pull Request в репозиторий прошивки `sc-bl`¹. Функциональность включается флагом сборки `PLF_HWCNT_DUMP`.

Основная сложность чтения 64-разрядного счётчика на 32-разрядном ядре состоит в том, что его половины читаются двумя отдельными инструкциями, между которыми младшее слово может переполниться и инкрементировать старшее. Чтобы исключить эту гонку, применяется стандартный приём с повторной выборкой старшего слова: старшее слово читается до и после чтения младшего, и при их расхождении чтение повторяется (листинг 1). Считанные значения `msycle`, `minstret` и `time` выводятся по UART в шестнадцатеричном виде с поясняющими подписями.

¹<https://github.com/syntacore/sc-bl/pull/3>

Листинг 1: Безопасное чтение и вывод аппаратных счётчиков в sc-bl

```
#ifdef PLF_HWCNT_DUMP
/* race-free read of a 64-bit counter split across lo/hi CSRs (RV32) */
#define read_counter64(dst, lo, hi) do{                                     |
    uint32_t _h, _l, _h2;                                              |
    do { _h = read_csr(hi); _l = read_csr(lo);                          |
        _h2 = read_csr(hi); } while (_h != _h2);                      |
    (dst) = ((uint64_t)_h << 32) | _l; }while(0)

static void cmd_dump_counters(void *arg)
{
    uint64_t cyc, inst, tim;

    read_counter64(cyc, mcycle, mcycleh);
    read_counter64(inst, minstret, minstreth);
    read_counter64(tim, time, timeh);

    uart_puts("HW counters:\n");
    uart_puts("  cycle:  "); uart_puthex64(cyc);  uart_putc('\n');
    uart_puts("  instret: "); uart_puthex64(inst); uart_putc('\n');
    uart_puts("  time:   "); uart_puthex64(tim);  uart_putc('\n');
}
#endif // PLF_HWCNT_DUMP
```

Полученный таким образом дамп даёт пару (`mcycle`, `minstret`) для каждого прогона тестовой программы, из которой и вычисляется базовое значение CPI, а счётчик `time` позволяет дополнительно соотнести результат с реальным временем исполнения.

Заключение

В рамках текущего этапа работы, посвящённого изучению подходов к реализации BPU и архитектуры `scr1`, были достигнуты следующие результаты, соответствующие поставленным в разделе 1 задачам:

- выполнен обзор подходов к реализации BPU — рассмотрены статические схемы предсказания («always taken/not taken», BTFNT) и динамические методы, от простых двухбитных счётчиков насыщения до двухуровневых адаптивных схем (gselect, gshare) и многотабличных предсказателей;
- проведён обзор открытых RISC-V-совместимых ядер с точки зрения реализации и внедрения BPU (Ibex, CV32E40P, VexRiscv, CVA6); на его основе показано, что в ядрах MCU-класса BPU либо отсутствует, либо реализуется как опциональный статический или простой динамический предсказатель, тогда как полноценные многотабличные схемы появляются лишь в ядрах прикладного класса;
- на основании количественных ориентиров из литературы и проведённого обзора предложены наиболее подходящие для `scr1` направления — лёгкий статический предсказатель либо минимальная динамическая схема bimodal/gshare с ограниченной ёмкостью РНТ, тогда как полноценный BPU с ВТВ и многотабличной структурой признан нецелесообразным без пересмотра бюджета площади ядра;
- в загрузочную прошивку `sc-bl` реализован дамп аппаратных счётчиков `scr1`, снимающий с ядра значения `mcycle`, `minstret` и `time`; тем самым создана измерительная инфраструктура, переводящая дальнейшую оценку BPU из плоскости качественных рассуждений в плоскость измеримых величин.

Настоящая работа рассчитана на несколько семестров; приведённые результаты относятся к текущему этапу, основной задачей которого было

погружение в архитектуру scr1 и подходы к реализации ВРУ, а также подготовка измерительной инфраструктуры. На следующем этапе планируется, опираясь на реализованный дамп счётчиков, снять профиль ветвлений реальных нагрузок scr1, синтезировать один из предложенных предсказателей и эмпирически оценить выигрыш в производительности в сопоставлении с приростом площади и энергопотребления. Программным результатом работы является дамп счётчиков scr1, оформленный как pull request в открытый репозиторий прошивки scbl.

Список литературы

- [1] URL: <https://riscv.org/members/> (дата обращения: 4 июня 2026 г.).
- [2] URL: <https://syntacore.com/products/scr1> (дата обращения: 4 июня 2026 г.).
- [3] URL: <https://github.com/syntacore/scr1> (дата обращения: 4 июня 2026 г.).
- [4] URL: <https://developer.arm.com/documentation/ddi0489/f/introduction/component-blocks/prefetch-unit> (дата обращения: 4 июня 2026 г.).
- [5] Customizing the branch predictor to reduce complexity and energy consumption / Michael C. Huang, Daniel Chaver, Luis Piñuel et al. // [IEEE Micro](#). — 2003. — Vol. 23, no. 5. — P. 12–25.
- [6] Efficient architectural exploration of TAGE branch predictor for embedded processors / Libo Huang, Qi Yu, Chaobing Zhou et al. // [Microelectronics Journal](#). — 2019. — Vol. 88. — P. 88–98.
- [7] From Swift to Mighty: A Cost-Benefit Analysis of Ibex and CV32E40P Regarding Application Performance, Power and Area / Noam Gallmann, Pirmin Vogel, Pasquale Davide Schiavone, Luca Benini // Workshop on Computer Architecture Research with RISC-V (CARRV). — 2021.
- [8] Kanamori Takuto, Miyazaki Hiromu, Kise Kenji. RVCOREP-32IC: A High-Performance RISC-V Soft Processor with an Efficient Fetch Unit Supporting the Compressed Instructions // International Conference on Field-Programmable Technology (FPT). — 2020. — arXiv:2011.11246.
- [9] Power-Aware Branch Prediction: Characterization and Design /

- Dharmesh Parikh, Kevin Skadron, Yan Zhang, Mircea R. Stan // [IEEE Transactions on Computers](#). — 2004. — Vol. 53, no. 2. — P. 168–186.
- [10] Shen John P, Lipasti Mikko H. Modern Processor Design: Fundamentals of Superscalar Processors. — Waveland Press, 2013.
- [11] SpinalHDL. VexRiscv: A FPGA-Friendly 32-Bit RISC-V CPU Implementation. — <https://github.com/SpinalHDL/VexRiscv>. — 2024. — Accessed: 2026-06-13.
- [12] Zaruba Florian, Benini Luca. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology // [IEEE Transactions on Very Large Scale Integration \(VLSI\) Systems](#). — 2019. — Vol. 27, no. 11. — P. 2629–2640.
- [13] lowRISC. Ibex RISC-V Core — Documentation and Source Repository. — <https://github.com/lowRISC/ibex>. — 2024. — Accessed: 2026-06-13.
- [14] sc-bl: Syntacore RISC-V bootloader. — URL: <https://github.com/syntacore/sc-bl> (дата обращения: 4 июня 2026 г.).